MINI PROJECT REPORT

On

"Implementation of Huffman Encoding using GPU Acceleration"

A Report Submitted for a mini project for: Laboratory Practice-V in 8th Semester Computer Engineering.

Fourth Year (COMPUTER ENGINEERING)

Academic Year 2025-26

Submitted by-

Jenil Girish Rathod

Roll No.: B22009 Div.: B

Zeal Education Society's

Zeal College of Engineering & Research Narhe,

Pune – 411041

Department of Computer Engineering

Zeal Education Society's

Zeal College of Engineering & Research Department of Computer Engineering

CERTIFICATE

Certified that the project entitled **"Implementation of Huffman Encoding using GPU Acceleration"** is a bona fide work carried out by **Jenil Girish Rathod (B22009).** It is certified that all corrections/suggestions indicated for Internal Assignment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the Bachelor of Engineering Degree.

Prof. Reshma Yawalkar — Project Guide

Prof. A. V. Mote — H. O. D

ACKNOWLEDGEMENT

We take this opportunity to thank our project guide **Prof. Reshma Yawalkar** mam and Head of the department **Prof. A. V. Mote** mam for their valuable guidance and for providing all the necessary facilities, which were indispensable in the completion of this project report. We are also thankful to all the staff members of Computer Engineering Department for their valuable time, support, comments, suggestions and persuasion. We would also like to thank the institute for providing the required facilities, Internet access and important books.

INDEX

ABSTRACT

This project presents the implementation of Huffman Encoding using GPU acceleration to enhance the performance of data compression systems. Huffman encoding is a widely used lossless compression technique that assigns variable-length binary codes to input symbols based on their frequency of occurrence. While the traditional implementation of this algorithm is efficient, it becomes computationally intensive when processing large datasets due to its reliance on sequential CPU operations.

To overcome this limitation, this project adopts a hybrid computing approach that leverages the parallel processing capabilities of Graphics Processing Units (GPUs). The frequency calculation and encoding stages are parallelized using CUDA, enabling multiple data elements to be processed simultaneously. The Huffman tree construction, which involves sequential operations, is handled by the CPU to maintain algorithmic correctness.

The proposed system demonstrates significant improvement in execution time and scalability compared to conventional CPU-based implementations. By efficiently utilizing GPU resources, the project highlights the potential of parallel computing in optimizing classical algorithms. This work provides a practical understanding of GPU acceleration and its application in high-performance data compression systems.

SOFTWARE AND HARDWARE REQUIREMENT

Software Requirements:

1. **Programming Language:** Python 3.x
2. **Framework / UI:** Streamlit (for web-based interface)
3. **GPU Programming Library:** Numba CUDA (for parallel execution)
4. **Libraries Used:** NumPy, Numba, Streamlit
5. **IDE / Editor:** VS Code / PyCharm / Jupyter Notebook
6. **Operating System:** Windows / Linux / macOS

Hardware Requirements

1. **Processor (CPU)**:
    - **Minimum**: Intel Core i3 (or equivalent) or higher
    - **Recommended**: Intel Core i5/i7 or equivalent AMD processor
2. **Memory (RAM)**:
    - **Minimum**: 4 GB of RAM
    - **Recommended**: 8 GB of RAM or more
3. **Storage**:
    - **Minimum**: 2 GB of free disk space
    - **Recommended**: 5 GB of free disk space or more
4. **Display**:
    - **Minimum**: 1024x768 screen resolution
    - **Recommended**: 1920x1080 screen resolution (Full HD)
5. **Graphics Processing Unit (GPU)**:
    - **Minimum**: NVIDIA GPU with CUDA support
    - **Recommended**: NVIDIA GTX/RTX Series

INTRODUCTION

In the modern era of data-driven applications, efficient data compression techniques play a crucial role in reducing storage requirements and improving transmission speed. Huffman encoding is a widely used lossless data compression algorithm that assigns variable-length binary codes to symbols based on their frequency of occurrence. It is commonly used in file compression formats and communication systems due to its simplicity and effectiveness.

Traditional implementations of Huffman encoding are executed on the Central Processing Unit (CPU), where operations such as frequency calculation, tree construction, and encoding are performed sequentially. While this approach is efficient for small datasets, it becomes a performance bottleneck when handling large volumes of data due to limited parallelism.

With the advancement of parallel computing, Graphics Processing Units (GPUs) have emerged as powerful hardware capable of executing thousands of operations simultaneously. This project leverages GPU acceleration to optimize the performance of Huffman encoding. By utilizing CUDA through the Numba library in Python, the encoding process specifically the mapping of input symbols to their corresponding Huffman codes is parallelized across multiple GPU threads.

The system adopts a hybrid architecture in which the Huffman tree construction and codebook generation are performed on the CPU, while the computationally intensive encoding step is offloaded to the GPU. A user-friendly interface is developed using Streamlit, allowing users to input text, perform compression, and visualize results such as compression ratio, encoded output, and decoded verification.

Additionally, the system includes a fallback mechanism that automatically switches to CPU execution if GPU support is unavailable, ensuring reliability and portability across different environments.

This project demonstrates how classical algorithms like Huffman encoding can be enhanced using modern parallel computing techniques. It provides practical insight into GPU programming, hybrid system design, and performance optimization, making it a valuable study in high-performance computing and real-world software implementation.

PROBLEM STATEMENT

Traditional Huffman encoding relies on CPU-based sequential processing, which leads to increased execution time and poor scalability for large datasets. There is a need to improve performance by utilizing GPU-based parallel processing while maintaining accurate and efficient lossless compression.

OBJECTIVE

- To implement the Huffman Encoding algorithm for text compression
- To design a hybrid system combining CPU and GPU processing
- To utilize GPU parallelism (CUDA via Numba) for faster encoding
- To develop an interactive user interface using Streamlit
- To ensure correctness through lossless compression and decompression
- To compare performance between CPU and GPU execution
- To handle fallback execution when GPU is unavailable

OUTCOME

- Successful implementation of Huffman encoding with GPU acceleration
- Reduction in execution time for encoding operations
- Efficient utilization of parallel processing capabilities of GPU
- Accurate compression and decompression (lossless behavior verified)
- Dynamic display of compression metrics such as size, bits, and ratio
- A functional web-based interface for user interaction and testing
- A scalable and reliable system with CPU fallback support

IMPLEMENTATION CODE

Technologies Used:

- Frontend: Streamlit
- Backend: Python with Numba

- **Backend Logic**

```
from __future__ import annotations

from collections import Counter
from dataclasses import dataclass
import base64
import heapq
from typing import Dict, List, Optional, Tuple

import numpy as np

try:
    from numba import cuda
except Exception:  # pragma: no cover - numba may not be installed yet
    cuda = None

@dataclass
class CompressionResult:
    original_text: str
    original_bytes: bytes
    encoded_bytes: bytes
    total_encoded_bits: int
    compression_ratio: float
    codebook: Dict[int, str]
    used_gpu: bool

@dataclass
class _Node:
    freq: int
    symbol: Optional[int] = None
    left: Optional["_Node"] = None
    right: Optional["_Node"] = None

def _build_huffman_tree(data: bytes) -> Optional[_Node]:
    if not data:
        return None

    frequency = Counter(data)
    heap: List[Tuple[int, int, _Node]] = []
    tie_breaker = 0
```

```
    for symbol, freq in frequency.items():
        heapq.heappush(heap, (freq, tie_breaker, _Node(freq=freq, symbol=symbol)))
        tie_breaker += 1

    if len(heap) == 1:
        only = heap[0][2]
        return _Node(freq=only.freq, left=only)

    while len(heap) > 1:
        f1, _, n1 = heapq.heappop(heap)
        f2, _, n2 = heapq.heappop(heap)
        merged = _Node(freq=f1 + f2, left=n1, right=n2)
        heapq.heappush(heap, (merged.freq, tie_breaker, merged))
        tie_breaker += 1

    return heap[0][2]

def _generate_codes(root: Optional[_Node]) -> Dict[int, str]:
    if root is None:
        return {}

    codes: Dict[int, str] = {}

    def walk(node: _Node, prefix: str) -> None:
        if node.symbol is not None:
            codes[node.symbol] = prefix if prefix else "0"
            return
        if node.left is not None:
            walk(node.left, prefix + "0")
        if node.right is not None:
            walk(node.right, prefix + "1")

    walk(root, "")
    return codes

def _codebook_to_lookup(codebook: Dict[int, str]) -> Tuple[np.ndarray, np.ndarray]:
    bits_lookup = np.zeros(256, dtype=np.uint64)
    len_lookup = np.zeros(256, dtype=np.uint8)

    for symbol, bitstring in codebook.items():
        value = 0
        for bit in bitstring:
            value = (value << 1) | (1 if bit == "1" else 0)
        bits_lookup[symbol] = np.uint64(value)
        len_lookup[symbol] = np.uint8(len(bitstring))

    return bits_lookup, len_lookup

if cuda is not None:
```

```
    @cuda.jit
    def _encode_kernel(symbols, bits_lut, lens_lut, out_bits, out_lens):
        i = cuda.grid(1)
        if i < symbols.size:
            sym = symbols[i]
            out_bits[i] = bits_lut[sym]
            out_lens[i] = lens_lut[sym]

def _encode_symbols_gpu_or_cpu(data: bytes, codebook: Dict[int, str]) ->
Tuple[np.ndarray, np.ndarray, bool]:
    symbols = np.frombuffer(data, dtype=np.uint8)
    bits_lut, lens_lut = _codebook_to_lookup(codebook)

    if symbols.size == 0:
        return np.array([], dtype=np.uint64), np.array([], dtype=np.uint8), False

    if cuda is not None and cuda.is_available():
        d_symbols = cuda.to_device(symbols)
        d_bits_lut = cuda.to_device(bits_lut)
        d_lens_lut = cuda.to_device(lens_lut)

        d_out_bits = cuda.device_array(symbols.size, dtype=np.uint64)
        d_out_lens = cuda.device_array(symbols.size, dtype=np.uint8)

        threads = 256
        blocks = (symbols.size + threads - 1) // threads
        _encode_kernel[blocks, threads](d_symbols, d_bits_lut, d_lens_lut, d_out_bits,
d_out_lens)

        out_bits = d_out_bits.copy_to_host()
        out_lens = d_out_lens.copy_to_host()
        return out_bits, out_lens, True

    out_bits = bits_lut[symbols]
    out_lens = lens_lut[symbols]
    return out_bits.astype(np.uint64), out_lens.astype(np.uint8), False

def _pack_codes(codes: np.ndarray, lengths: np.ndarray) -> Tuple[bytes, int]:
    out = bytearray()
    bit_buffer = 0
    bit_count = 0

    for code, length in zip(codes, lengths):
        l = int(length)
        if l == 0:
            continue
        bit_buffer = (bit_buffer << l) | int(code)
        bit_count += l
```

```
        while bit_count >= 8:
            shift = bit_count - 8
            out.append((bit_buffer >> shift) & 0xFF)
            bit_buffer &= (1 << shift) - 1
            bit_count -= 8

    if bit_count > 0:
        out.append((bit_buffer << (8 - bit_count)) & 0xFF)

    total_bits = int(np.sum(lengths, dtype=np.uint64))
    return bytes(out), total_bits

def _to_bitstring(encoded_bytes: bytes, total_bits: int) -> str:
    if not encoded_bytes or total_bits == 0:
        return ""
    joined = "".join(f"{b:08b}" for b in encoded_bytes)
    return joined[:total_bits]

def compress_text(text: str) -> CompressionResult:
    source = text.encode("utf-8")
    if not source:
        return CompressionResult(
            original_text=text,
            original_bytes=b"",
            encoded_bytes=b"",
            total_encoded_bits=0,
            compression_ratio=0.0,
            codebook={},
            used_gpu=False,
        )

    tree = _build_huffman_tree(source)
    codebook = _generate_codes(tree)

    codes, lengths, used_gpu = _encode_symbols_gpu_or_cpu(source, codebook)
    encoded_bytes, total_bits = _pack_codes(codes, lengths)

    original_size = len(source)
    compressed_size = len(encoded_bytes)
    ratio = (compressed_size / original_size) if original_size else 0.0

    return CompressionResult(
        original_text=text,
        original_bytes=source,
        encoded_bytes=encoded_bytes,
        total_encoded_bits=total_bits,
        compression_ratio=ratio,
        codebook=codebook,
        used_gpu=used_gpu,
    )
```

```
def decompress_text(result: CompressionResult) -> str:
    if not result.original_bytes:
        return ""

    reverse = {code: symbol for symbol, code in result.codebook.items()}
    bits = _to_bitstring(result.encoded_bytes, result.total_encoded_bits)

    decoded = bytearray()
    cursor = ""
    for bit in bits:
        cursor += bit
        symbol = reverse.get(cursor)
        if symbol is not None:
            decoded.append(symbol)
            cursor = ""

    return decoded.decode("utf-8", errors="replace")

def codebook_rows(result: CompressionResult) -> List[Dict[str, str]]:
    if not result.codebook:
        return []

    frequency = Counter(result.original_bytes)
    rows = []

    for symbol, code in sorted(result.codebook.items(), key=lambda item: (len(item[1]),
item[1])):
        display = chr(symbol) if 32 <= symbol <= 126 else f"0x{symbol:02X}"
        rows.append(
            {
                "symbol": display,
                "byte": str(symbol),
                "frequency": str(frequency[symbol]),
                "code": code,
                "length": str(len(code)),
            }
        )

    return rows

def build_export_payload(result: CompressionResult) -> bytes:
    lines = [
        "Huffman GPU Mini Project Export",
        f"backend={'GPU' if result.used_gpu else 'CPU'}",
        f"original_size={len(result.original_bytes)}",
        f"compressed_size={len(result.encoded_bytes)}",
        f"total_bits={result.total_encoded_bits}",
        "codebook:",
    ]
```

```
    for symbol, code in sorted(result.codebook.items()):
        lines.append(f"{symbol}:{code}")

    lines.append("payload_base64:")
    lines.append(base64.b64encode(result.encoded_bytes).decode("ascii"))

    return "\n".join(lines).encode("utf-8")
```

Frontend/UI

```
import streamlit as st

from huffman_gpu import build_export_payload, codebook_rows, compress_text,
decompress_text

st.set_page_config(page_title="GPU Huffman Encoder", page_icon="H", layout="wide")

st.title("Mini Project: Huffman Encoding on GPU")
st.caption("Built with Streamlit + CUDA (Numba). If CUDA is unavailable, it automatically
falls back to CPU.")

left, right = st.columns([3, 2])

with left:
    sample = """GPU-based Huffman compression demo text. Change this input and click
Encode."""
    text = st.text_area("Input text", value=sample, height=220)
    encode_clicked = st.button("Encode", type="primary", use_container_width=True)

with right:
    st.subheader("How it works")
    st.markdown(
        "\n".join(
            [
                "1. Build Huffman tree and variable-length codebook on CPU.",
                "2. Send input bytes and lookup tables to GPU.",
                "3. Parallel kernel maps each symbol to (code bits, code length).",
                "4. CPU packs variable-length bitstream into final compressed bytes.",
            ]
        )
    )

if encode_clicked:
    result = compress_text(text)
    st.session_state["huffman_result"] = result

if "huffman_result" in st.session_state:
    result = st.session_state["huffman_result"]
```

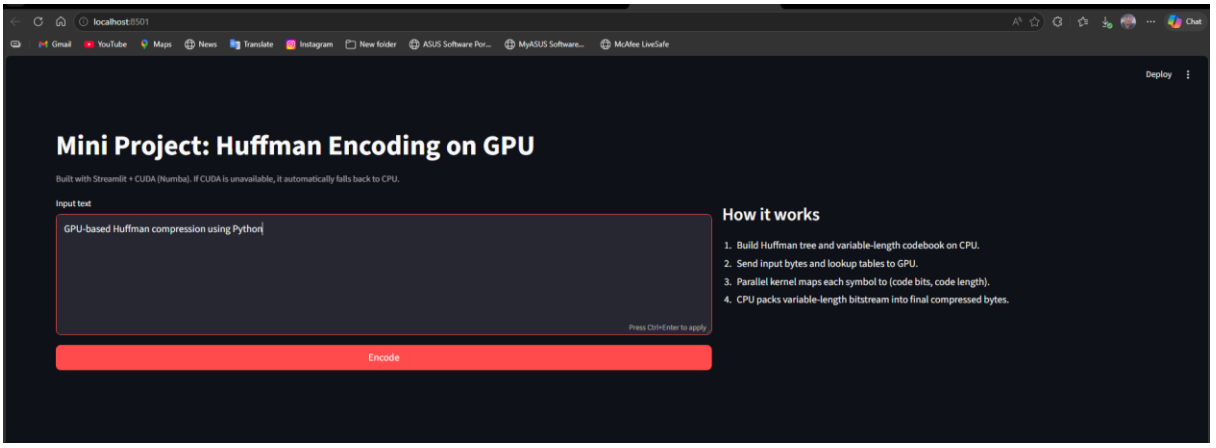
```
st.subheader("Compression Metrics")
m1, m2, m3, m4 = st.columns(4)
m1.metric("Original (bytes)", f"{len(result.original_bytes)}")
m2.metric("Compressed (bytes)", f"{len(result.encoded_bytes)}")
m3.metric("Total Bits", f"{result.total_encoded_bits}")
m4.metric("Ratio", f"{result.compression_ratio:.3f}")

st.info(f"Execution backend: {'GPU (CUDA)' if result.used_gpu else 'CPU fallback'}")

st.subheader("Codebook")
rows = codebook_rows(result)
if rows:
    st.table(rows)
else:
    st.write("No symbols to display for empty input.")

st.subheader("Encoded Output Preview")
st.code(result.encoded_bytes.hex()[:800] or "(empty)", language="text")

decoded = decompress_text(result)
st.subheader("Decoded Text")
st.text_area("Decoded result", value=decoded, height=180)

if decoded == result.original_text:
    st.success("Decode verification passed. Original text restored correctly.")
else:
    st.error("Decode verification failed. Decoded text differs from original.")

export_blob = build_export_payload(result)
st.download_button(
    label="Download compressed report",
    data=export_blob,
    file_name="huffman_gpu_export.txt",
    mime="text/plain",
    use_container_width=True,
)
```

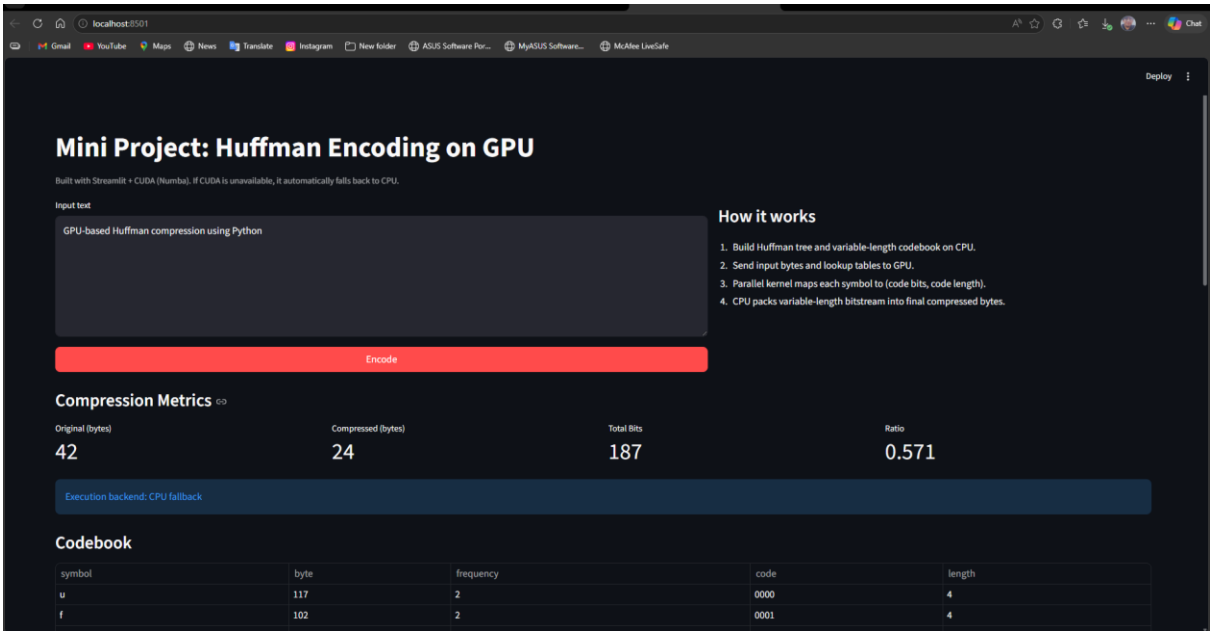
Test Cases

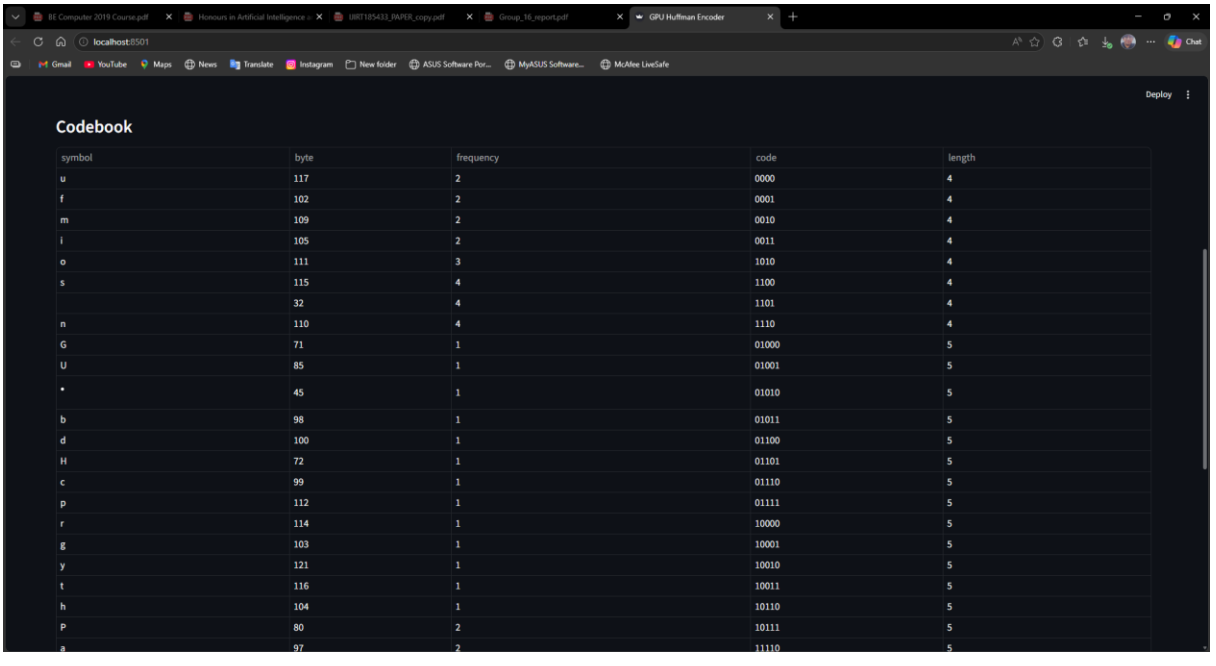
The system is tested for correctness, performance, and reliability across different modules including compression, GPU execution, and user interface.

Module Name	Test Scenario ID	Test Scenario Name	Test Cases	Test Case ID	Test Case Description
Compression Module	TS_001	Validate Input Handling	2	TC_001	Verify system accepts valid text input
				TC_002	Verify system handles empty input correctly
Compression Module	TS_002	Huffman Encoding	2	TC_003	Verify correct generation of Huffman codes
				TC_004	Verify compressed output is generated
GPU Module	TS_003	GPU Execution	2	TC_005	Verify encoding runs on GPU when available
				TC_006	Verify fallback to CPU when GPU is unavailable
Compression Module	TS_004	Compression Accuracy	2	TC_007	Verify compression ratio is calculated correctly
				TC_008	Verify encoded bit count is accurate
Decompression Module	TS_005	Lossless Recovery	2	TC_009	Verify decoded output matches original input
				TC_010	Verify no data loss during compression/decompression
UI Module	TS_006	User Interface	2	TC_011	Verify input text area and encode button functionality
				TC_012	Verify display of compression metrics and output
System Module	TS_007	Performance	2	TC_013	Verify faster execution using GPU compared to CPU
				TC_014	Verify system handles large input efficiently

OUTPUT


localhost:8501
Deploy
Mini Project: Huffman Encoding on GPU
Built with Streamlit + CUDA (Numba). If CUDA is unavailable, it automatically falls back to CPU.
Input text
GPU-based Huffman compression using Python
Press Ctrl+Enter to apply
Encode
How it works
1. Build Huffman tree and variable-length codebook on CPU.
2. Send input bytes and lookup tables to GPU.
3. Parallel kernel maps each symbol to (code bits, code length).
4. CPU packs variable-length bitstream into final compressed bytes.


localhost:8501
Deploy
Mini Project: Huffman Encoding on GPU
Built with Streamlit + CUDA (Numba). If CUDA is unavailable, it automatically falls back to CPU.
Input text
GPU-based Huffman compression using Python
Encode
How it works
1. Build Huffman tree and variable-length codebook on CPU.
2. Send input bytes and lookup tables to GPU.
3. Parallel kernel maps each symbol to (code bits, code length).
4. CPU packs variable-length bitstream into final compressed bytes.
Compression Metrics
Original (bytes) 42
Compressed (bytes) 24
Total Bits 187
Ratio 0.571
Execution backend: CPU fallback
Codebook
symbol byte frequency code length
u 117 2 0000 4
f 102 2 0001 4


GPU Huffman Encoder
localhost:8501
Deploy
Codebook
symbol byte frequency code length
u 117 2 0000 4
f 102 2 0001 4
m 109 2 0010 4
i 105 2 0011 4
o 111 3 1010 4
s 115 4 1100 4
32 4 1101 4
n 110 4 1110 4
G 71 1 01000 5
U 85 1 01001 5
• 45 1 01010 5
b 98 1 01011 5
d 100 1 01100 5
H 72 1 01101 5
c 99 1 01110 5
p 112 1 01111 5
r 114 1 10000 5
g 103 1 10001 5
y 121 1 10010 5
t 116 1 10011 5
h 104 1 10110 5
P 80 2 10111 5
a 97 2 11110 5

CONCLUSION

This project successfully implements Huffman Encoding using a hybrid CPU–GPU approach to improve the performance of data compression. By utilizing GPU acceleration through CUDA (Numba), the encoding process is parallelized, reducing execution time compared to traditional CPU-based methods. The system efficiently combines CPU-based operations such as tree construction and decoding with GPU-based parallel encoding, ensuring both accuracy and optimized performance.

Additionally, the project provides a user-friendly interface using Streamlit, allowing users to input data, view compression results, and verify output correctness. The inclusion of a CPU fallback mechanism ensures reliability across systems without GPU support. Overall, the project demonstrates how parallel computing can enhance classical algorithms and offers practical insights into high-performance computing and real-world application development.

REFERENCES: -

Source Code: - https://hpc-mini-project.streamlit.app/

1. NVIDIA Corporation, CUDA Programming Guide, Available at: https://docs.nvidia.com/cuda/
2. Numba Documentation, CUDA Programming with Python, Available at: https://numba.readthedocs.io/
3. GeeksforGeeks, Huffman Coding (Greedy Algorithm), Available at: https://www.geeksforgeeks.org/huffman-coding-greedy-algo-3/
4. Streamlit Documentation, Streamlit Web Framework, Available at: https://docs.streamlit.io/
5. NumPy Documentation, Available at: https://numpy.org/doc/
6. W3Schools, Data Compression Basics, Available at: https://www.w3schools.com/
7. David A. Huffman, A Method for the Construction of Minimum-Redundancy Codes, 1952